:main: name

:main: This is equivalent to the --main command-line parameter, setting the initial page displayed for this documentation.

:stopdoc: / :startdoc:

```
<p>
  :stopdoc: :startdoc: This stops and starts adding new
  documentation elements to the current container. For
  example, if a class has a number of constants that you don't
  want to document, put a <ic>:stopdoc:</ic> before the first
  and a <ic>:startdoc:</ic> after the last. If you don't
  specify a <ic>:startdoc:</ic> by the end of the container,
  this disables documentation for the entire class or module.
</p>
```

:enddoc:

```
<p>
  :enddoc: This documents nothing further at the current lexical level.
</p>
```

Running RDoc

RDoc can be run from the command line, like this:

```
$ rdoc OPTIONS FILENAMES
```

Type rdoc --help for an up-to-date option summary.

Files are parsed, and the information they contain collected, before any output is produced. This allows cross-references between all files to be resolved. If a name is a directory, it is traversed. If no names are specified, all Ruby files in the current directory (and subdirectories) are processed.

A typical use may be to generate documentation for a package of Ruby source (such as RDoc itself):

```
$ rdoc
```

This command generates HTML documentation for the files in and below the current directory. These will be stored in a documentation tree starting in the subdirectory doc/.

RDoc uses file extensions to determine how to process each file. Filenames with rb and rbw extensions are assumed to be Ruby source. Filenames with c, h, or cpp extension are parsed as C files. Files with md and markdown as Markdown. A file named ChangeLog will be parsed as a change log. All other files are assumed to contain just markup (with or without leading # comment markers). If directory names are passed to RDoc, they are scanned recursively for source files only. To include non-source files such as READMEs in the documentation process, their names must be given explicitly on the command line.

When writing a Ruby library, you often have some source files that implement the public interface, but the majority are internal and of no interest to the readers of your documentation. In these cases, construct a .document file in each of your project's directories. If RDoc enters a directory containing a .document file, it will process only the files in that directory whose names match one of the lines in that file. Each line in the file can be a filename, a directory